# Implementierung und Analyse eines Feed-Forward-Netzes in Java am Beispiel des XOR-Problems

28. Oktober 2026

## Inhaltsverzeichnis


**1 Einleitung** **3**
- 1.1 Aufgabenstellung . . . 3
- 1.2 Zielsetzung . . . 3
- 1.3 Aufgabenkontext . . . 3
- 1.4 Externe Vorgaben . . . 3
- 1.5 Ermessensentscheidungen . . . 3

**2 Theoretische Grundlagen** **5**
- 2.1 Was ist ein künstliches neuronales Netz? . . . 5
- 2.2 Backpropagation . . . 5
  - 2.2.1 Vorwärtsrechnung und Fehlerfunktion . . . 5
  - 2.2.2 Fehlersignale der Ausgabe- und verborgenen Schichten . . . 5
  - 2.2.3 Gewichtsänderung und Momentum . . . 6

**3 Methodik** **7**
- 3.1 Architektur und Basis-Implementierung des Feed-Forward-Netzes . . . 7
- 3.2 Umsetzung der Backpropagation in Java . . . 7
  - 3.2.1 Gespeicherte Werte aus dem Vorwärtsdurchlauf . . . 7
  - 3.2.2 Berechnung der Fehlersignale . . . 7
  - 3.2.3 Bias, Momentum und verwendete Fehlerfunktion . . . 8
  - 3.2.4 Einbindung in die Trainingsschleife . . . 9

**4 Ergebnisse** **10**
- 4.1 Korrektheitsprüfung der Backpropagation . . . 10
- 4.2 Performance und Trainingsverhalten . . . 10

**5 Diskussion** **12**
- 5.1 Einordnung der Backpropagation-Implementierung . . . 12

**6 Fazit und Ausblick** **13**
- 6.1 Zusammenfassung . . . 13
- 6.2 Persönliches Fazit . . . 13

# Arbeitsteilung

| Teammitglied | Programmierung & Testing | Dokumentation (Projektbuch) | Organisation & Sonstiges |
|---|---|---|---|
| **Liam Kelders** | FFN-Basisarchitektur (Nodes, Layers, Activation) & Loss-Funktion | Kapitel 1 (Einleitung) & LaTeX-Grundgerüst | Verwaltung der Projektdokumentation auf LEA |
| **Justus Braun** | Implementierung des Backpropagation-Algorithmus | Kapitel 2 (Theoretische Grundlagen) & Kapitel 3 (Methodik) | Fachliche Aufbereitung der Backprop-Konzepte |
| **Aaron Strohm** | Co-Entwicklung Delta-Regel & Testen des Lernverhaltens | Kapitel 6 (Fazit & Ausblick) | Konzeption & Erstellung der Abschlusspräsentation |
| **Anton Niemann** | Repository-Setup, Code-Integration & Loss-History Logging | Kapitel 4 (Ergebnisse) & Kapitel 5 (Diskussion), Feinschliff | Datenanalyse & Visualisierung der Diagramme |

# 1 Einleitung

## 1.1 Aufgabenstellung

Im Rahmen der Lehrveranstaltung *Lernen in natürlichen und künstlichen neuronalen Netzen* wurde die Aufgabe gestellt, ein einfaches Feed-Forward-Netz mit überwachter Lernstrategie eigenständig zu implementieren. Das Netz soll in der Lage sein, dreidimensionale Eingabemuster (drei kontinuierliche Merkmale) einer von drei vorgegebenen Klassen zuzuordnen. Die Umsetzung umfasst die vollständige Pipeline von der Datenaufbereitung über die Vorwärts- und Rückwärtspropagation bis hin zur Auswertung der Klassifikationsleistung. Der vollständige Quellcode sowie die Dokumentation sind im öffentlichen Repository unter https://github.com/Kolop1012/LikunnN_Feed-Forward-Netz_A1 einsehbar.

Der vorliegende Bericht dokumentiert die tatsächlich implementierte und ausgewertete Teilaufgabe: das binäre XOR-Problem mit zwei Eingängen, einer Ausgabe und der Netzarchitektur 2–4–1. Die in der ursprünglichen Aufgabenbeschreibung geforderte Klassifikation dreidimensionaler Muster in drei Klassen wurde in dieser Umsetzung nicht realisiert. Methodik und Ergebnisse beziehen sich daher ausschließlich auf das XOR-Problem; eine Erfüllung der ursprünglichen Klassifikationsvorgabe wird nicht beansprucht.

## 1.2 Zielsetzung

Ziel des Projekts ist es, ein funktionsfähiges, aber bewusst schlank gehaltenes neuronales Netz zu entwickeln, das den gesamten Lernzyklus (Init, Forward-Pass, Fehlerberechnung, Backpropagation, Gewichts-Update) selbst implementiert ohne auf Frameworks zurückzugreifen. Dadurch soll ein vertieftes Verständnis der zugrundeliegenden mathematischen Zusammenhänge erreicht werden.

## 1.3 Aufgabenkontext

Das Projekt entstand im Rahmen des Moduls *Lernen in natürlichen und künstlichen neuronalen Netzen* im Studiengang *Informatik*. Es wurde als Gruppenarbeit über einen Zeitraum von vier Wochen bearbeitet. Die Abgabe umfasst sowohl den lauffähigen Code als auch diesen Projektbericht. Die Bewertung orientiert sich an der im SW02 veröffentlichten Bewertungsmatrix (formale Vorgaben, wissenschaftlicher Schreibstil, Nachvollziehbarkeit der Umsetzung, Diskussion der Vor- und Nachteile).

## 1.4 Externe Vorgaben

Folgende Vorgaben wurden von Seiten der Lehrveranstaltung gemacht und im Projekt berücksichtigt:

- Implementierung eines *Feed-Forward-Netzes* mit mindestens einer versteckten Schicht.
- Überwachtes Lernen mittels *Backpropagation*.
- Klassifikation von drei vorgegebenen Klassen.
- Seitenumfang des Berichts: 10 Seiten (± 1).
- Schrift: Times (12 pt) oder Arial (11 pt), Zeilenabstand 1,5, Rand 2,5 cm, Blocksatz mit Silbentrennung.
- Verpflichtende Literaturrecherche und Angabe aller Quellen.

## 1.5 Ermessensentscheidungen

Über die externen Vorgaben hinausgehend wurden folgende Entscheidungen eigenständig im Projektteam getroffen:

- Verwendung der *halben Fehlerquadratsumme* als Fehlerfunktion (statt Kreuzentropie), da sie mathematisch einfacher herzuleiten ist und für die kleine Netzarchitektur ausreichend konvergiert.
- Wahl der *logistischen Sigmoidfunktion* als Aktivierungsfunktion in allen Schichten (statt ReLU), weil sie differenzierbar ist und die Ausgaben natürlicherweise auf $(0, 1)$ beschränkt bleiben.
- Feste Lernrate von $\eta = 0{,}1$ mit manuell gewählter Epochengrenze (kein Early Stopping), um den Lernverlauf im Bericht transparent dokumentieren zu können.
- Verwendung aller vier XOR-Muster als Trainings- und Bewertungsdaten. Ein separater Testdatensatz wurde nicht eingesetzt; die Ergebnisse erlauben daher keine Aussage zur Generalisierung auf unbekannte Daten.

# 2 Theoretische Grundlagen

## 2.1 Was ist ein künstliches neuronales Netz?

Ein künstliches neuronales Netz beschreibt eine parametrisierte Abbildung von Eingabewerten auf Ausgabewerte. Bei einem mehrschichtigen Feed-Forward-Netz werden die Eingaben schrittweise durch eine oder mehrere verborgene Schichten zur Ausgabeschicht verarbeitet. Jedes verarbeitende Neuron bildet eine gewichtete Summe seiner Eingaben, ergänzt einen Bias und wendet eine Aktivierungsfunktion an. Beim überwachten Lernen stehen zu den Eingaben gewünschte Ausgaben zur Verfügung. Die Gewichte und Bias-Werte werden anhand dieser Beispiele angepasst [4, Folien 4 und 6].

## 2.2 Backpropagation

Für die Ausgabeschicht kann die berechnete Ausgabe unmittelbar mit dem Zielwert verglichen werden. Für die verborgenen Neuronen fehlen solche Zielwerte. Backpropagation löst dieses Problem, indem der Einfluss der Netzparameter auf den Ausgabefehler mithilfe der Kettenregel rückwärts durch das Netz berechnet wird. Dabei entstehen keine zusätzlichen Trainingsziele für die verborgenen Schichten, sondern Ableitungen der Fehlerfunktion. Rumelhart, Hinton und Williams beschreiben, wie auf dieser Grundlage auch verborgene Neuronen für die Aufgabe geeignete interne Repräsentationen erlernen können [3]. Backpropagation bezeichnet dabei die Berechnung des Gradienten; die anschließende Parameteränderung erfolgt durch ein Optimierungsverfahren wie den Gradientenabstieg [1, Abschn. 6.5].

### 2.2.1 Vorwärtsrechnung und Fehlerfunktion

Im Folgenden bezeichnet $l = 0$ die Eingabeschicht und $l = L$ die Ausgabeschicht. Die Schicht $l$ besitzt $n_l$ Neuronen. Das Gewicht $w_{ij}^{(l)}$ verbindet Neuron $j$ der vorherigen Schicht mit Neuron $i$ der aktuellen Schicht; $b_i^{(l)}$ ist dessen Bias. Für eine Eingabe $\boldsymbol{x}$ gilt $\boldsymbol{a}^{(0)} = \boldsymbol{x}$. Die Vorwärtsrechnung lautet

$$z_i^{(l)} = \sum_{j=1}^{n_{l-1}} w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}, \qquad a_i^{(l)} = f\left(z_i^{(l)}\right). \tag{1}$$

Hierbei ist $z_i^{(l)}$ die gewichtete Eingabesumme vor der Aktivierung und $a_i^{(l)}$ der Aktivierungswert. Diese Notation unterscheidet somit zwischen dem Argument der Aktivierungsfunktion und ihrem Ergebnis [2, Kap. 2].

Passend zur Implementierung wird für ein einzelnes Trainingsmuster mit Zielvektor $\boldsymbol{y}$ die halbe Fehlerquadratsumme verwendet:

$$E = \frac{1}{2} \sum_{i=1}^{n_L} \left(y_i - a_i^{(L)}\right)^2. \tag{2}$$

Der Faktor $1/2$ vereinfacht die Ableitung, da er den beim Differenzieren des Quadrats entstehenden Faktor 2 aufhebt. Daraus ergibt sich $\partial E / \partial a_i^{(L)} = a_i^{(L)} - y_i$. Es wird hier über die Ausgabeneuronen summiert und nicht gemittelt. Für einen Datensatz mit $N$ Mustern wird zur Bewertung dagegen der mittlere Musterfehler $\overline{E} = N^{-1} \sum_{p=1}^{N} E_p$ betrachtet, wobei $E_p$ den Fehler des Musters $p$ bezeichnet. Einzelner Musterfehler und mittlerer Datensatzfehler sind daher zu unterscheiden.

### 2.2.2 Fehlersignale der Ausgabe- und verborgenen Schichten

Entsprechend der Delta-Regel der Lehrveranstaltung und dem Vorzeichen im Quellcode wird das Fehlersignal definiert als

$$\delta_i^{(l)} := -\frac{\partial E}{\partial z_i^{(l)}}. \tag{3}$$

Damit ergibt sich für ein Ausgabeneuron durch Anwendung der Kettenregel

$$\delta_i^{(L)} = -\frac{\partial E}{\partial a_i^{(L)}} \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}} = \left(y_i - a_i^{(L)}\right) f'\left(z_i^{(L)}\right). \tag{4}$$

Die Ausgabedifferenz wird also mit der lokalen Ableitung der Aktivierungsfunktion gewichtet [4, Folie 10]. Nielsen verwendet hingegen $\delta = \partial E / \partial z$ und damit das entgegengesetzte Vorzeichen. Beide Konventionen führen bei entsprechendem Vorzeichen des Gewichtsupdates zum gleichen Gradientenabstieg [2, Kap. 2].

Ein verborgenes Neuron beeinflusst den Fehler über alle Neuronen der nächsten Schicht. Die Kettenregel liefert deshalb eine Summe über diese Nachfolger:

$$\delta_i^{(l)} = f'\left(z_i^{(l)}\right) \sum_{k=1}^{n_{l+1}} w_{ki}^{(l+1)} \delta_k^{(l+1)}, \qquad l = L-1, \ldots, 1. \tag{5}$$

Jedes nachfolgende Fehlersignal geht mit dem zugehörigen Verbindungsgewicht ein. Anschließend wird die Ableitung des betrachteten Neurons berücksichtigt. Die Berechnung beginnt somit an der Ausgabeschicht und setzt sich bis zur ersten verborgenen Schicht fort [4, Folien 10 und 12].

In der Demo wird die Sigmoid-Funktion eingesetzt. Für sie gilt

$$f(z) = \frac{1}{1 + \exp(-z)}, \qquad f'(z) = f(z)\big(1 - f(z)\big). \tag{6}$$

Ihre Ableitung ist bei stark positiven oder negativen Eingabesummen klein. Dadurch können auch bei einer deutlichen Abweichung vom Zielwert nur kleine Fehlersignale entstehen; das Lernen kann sich verlangsamen [2, Kap. 2].

### 2.2.3 Gewichtsänderung und Momentum

Aus Gleichung (1) und der Definition des Fehlersignals folgt

$$\frac{\partial E}{\partial w_{ij}^{(l)}} = -\delta_i^{(l)} a_j^{(l-1)}, \qquad \frac{\partial E}{\partial b_i^{(l)}} = -\delta_i^{(l)}. \tag{7}$$

Ein Schritt entgegen dem Gradienten mit Lernrate $\eta > 0$ lautet daher

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} + \eta\, \delta_i^{(l)} a_j^{(l-1)}, \qquad b_i^{(l)} \leftarrow b_i^{(l)} + \eta\, \delta_i^{(l)}. \tag{8}$$

Das Pluszeichen ist eine Folge der gewählten Definition von $\delta$ und entspricht der Lernregel der Vorlesung [4, Folie 10]. Der Bias lässt sich als Gewicht einer konstanten Eingabe 1 auffassen und mit derselben Regel aktualisieren. Alle Fehlersignale eines Musters müssen dabei mit den Gewichten berechnet werden, die auch im zugehörigen Vorwärtsdurchlauf verwendet wurden.

Optional berücksichtigt das Projekt die vorherige Parameteränderung durch Momentum. Für einen beliebigen Netzparameter $\theta$, also ein Gewicht oder einen Bias, wird im Lernschritt $t$ berechnet:

$$\Delta\theta_t = \mu\, \Delta\theta_{t-1} - \eta_t \frac{\partial E_t}{\partial \theta}, \qquad \theta_{t+1} = \theta_t + \Delta\theta_t. \tag{9}$$

Dabei bezeichnet $E_t$ den Fehler des aktuell bearbeiteten Musters, $\mu \in [0, 1)$ den Momentum-Faktor und $\Delta\theta_{t-1}$ die zuletzt gespeicherte Änderung; diese ist vor dem ersten Schritt null. Für $\mu = 0$ entsteht der gewöhnliche Gradientenabstieg. Momentum kann das Lernen bei wiederholt ähnlicher Gradientenrichtung beschleunigen und Schwingungen verringern [1, Abschn. 8.3.2].

# 3 Methodik

## 3.1 Architektur und Basis-Implementierung des Feed-Forward-Netzes

Um die Zielsetzung eines tiefgreifenden Verständnisses für neuronale Netze zu erfüllen, wurde die gesamte Architektur in nativem Java ohne externe Machine-Learning-Bibliotheken entwickelt. Die Logik gliedert sich primär in die Komponenten `FFNeuralNetwork`, `Layer` und `ActivationFunction`.

**Beschreibung der Umsetzung:** Das Netz (`FFNeuralNetwork`) verwaltet ein Array von `Layer`-Objekten. Anstatt jedes Neuron als eigenständiges Objekt zu modellieren, kapselt jede Schicht (`Layer`) die Eigenschaften all ihrer Neuronen in Arrays: Sie speichert die aktuelle Anzahl der Neuronen, die Aktivierungswerte sowie die zugehörigen eingehenden Gewichtsmatrizen (`weights`) und Bias-Werte (`biases`). Der Vorwärtsdurchlauf (`feedForward`) iteriert sequenziell über diese Schichten. Jede Schicht berechnet die gewichtete Summe der Eingaben der vorherigen Schicht, addiert den Bias und wendet über `ActivationFunction` die zugewiesene nicht-lineare Aktivierungsfunktion an.

**Begründung der Umsetzung:** Die Entscheidung für eine objektorientierte Kapselung auf Schicht-Ebene anstelle einer Modellierung einzelner Neuronen-Objekte ist architektonisch begründet: Sie nähert den Code an die zugrundeliegenden mathematischen Vektor- und Matrixoperationen an und vermeidet einen massiven Objekt-Overhead im Speicher. Die iterative Array-Verarbeitung erhöht zudem die Cache-Lokalität und Laufzeiteffizienz. Die Auslagerung der Aktivierungsfunktionen in eine eigenständige Struktur dient der Modularität. So können verschiedene Aktivierungen (z. B. Sigmoid, ReLU) einfach zugewiesen und ausgetauscht werden, ohne die Kernlogik der Schichten anpassen zu müssen.

## 3.2 Umsetzung der Backpropagation in Java

Die Backpropagation nutzt dieselben drei Kernklassen. `FFNeuralNetwork` steuert die Reihenfolge der Berechnungen, `Layer` berechnet die Fehlersignale und aktualisiert die Parameter, während `ActivationFunction` die benötigten mathematischen Ableitungen bereitstellt. Die folgende Beschreibung bezieht sich auf den Quellcode unter `src/main/java/NeuralNetwork/`.

### 3.2.1 Gespeicherte Werte aus dem Vorwärtsdurchlauf

**Umsetzung:** Vor jedem Lernschritt wird `feedForward` mit dem aktuellen Trainingsmuster aufgerufen. Die Methode `Layer.forward` speichert die Aktivierungen der Vorgängerschicht in `lastInputs` und die gewichteten Eingabesummen in `lastRawInputs`. Damit stehen für die Rückwärtsrechnung die Größen $a_j^{(l-1)}$, $z_i^{(l)}$ und $a_i^{(l)}$ aus Gleichung (1) direkt zur Verfügung. Die Ableitung wird durch `applyDerivative(lastRawInputs[i])` ausgewertet (das Argument ist die Eingabesumme $z_i^{(l)}$ vor der Aktivierung).

**Begründung:** Diese Caching-Strategie ist eine bewusste Designentscheidung. Indem die Zwischenergebnisse des Vorwärtsdurchlaufs (insbesondere $z_i^{(l)}$) im Speicher gehalten werden, wird enorm viel Rechenzeit gespart. Eine Neuberechnung all dieser Werte während des Backpropagation-Schritts wäre höchst ineffizient und würde die Trainingslaufzeit drastisch verlängern. Der minimal erhöhte Speicherbedarf für `lastRawInputs` wird zugunsten der Performance in Kauf genommen.

### 3.2.2 Berechnung der Fehlersignale

**Umsetzung:** Die Methode `train` beginnt bei der Ausgabeschicht. `Layer.calculateDeltas` unterscheidet zwei Fälle: Liegen Zielwerte vor, wird pro Ausgabeneuron `(targets[i] - output) * derivative` berechnet (entspricht Gleichung (4)). Für eine verborgene Schicht werden stattdessen die Deltas und die Gewichtsmatrix der nächsten Schicht übergeben. Die Summe `nextLayerDeltas[k] * nextLayerWeights[k][j]` setzt die gewichtete Fehlerrückführung aus Gleichung (5) um. Die Gewichtsmatrix ist als `weights[i][j]` aufgebaut: Der erste Index

(`i`) bezeichnet das aktuelle Neuron, der zweite (`j`) ein Neuron der Vorgängerschicht. `nextLayerWeights[k][j]` bezeichnet somit die Verbindung vom aktuellen Neuron `j` zum nachfolgenden Neuron `k`.

**Begründung:** Die Wahl dieser spezifischen Array-Indizierung realisiert implizit die Multiplikation mit der transponierten Gewichtsmatrix. Es ist eine bewusste Optimierung, die *kein* zusätzliches Array für die transponierte Matrix anlegt, was Speicherkapazität spart und den Garbage Collector von Java entlastet. Die strikte Trennung von Delta-Berechnung und Gewichts-Update (siehe Listing 3.1) garantiert algorithmische Integrität, da ein Fehlersignal so niemals fälschlicherweise mit bereits aktualisierten Gewichten berechnet wird.

```
double[][] deltas = new double[layers.length][];
deltas[lastIndex] = layers[lastIndex].calculateDeltas(null, null, targets);
for (int i = lastIndex - 1; i > 0; i--) {
        deltas[i] = layers[i].calculateDeltas(
        deltas[i + 1], layers[i + 1].getWeights(), null);
    }
for (int i = 1; i <= lastIndex; i++) {
        layers[i].updateWeights(deltas[i], learningRate, momentum);
    }
forwardReady = false;
```

Listing 1: Berechnungsreihenfolge in `FFNeuralNetwork.train` (Auszug aus der eigenen Implementierung).

Das Attribut `forwardReady` erzwingt architekturseitig, dass vor jedem Lernschritt ein gültiger Vorwärtsdurchlauf erfolgt. Ein erneuter Aufruf von `train` ohne vorheriges `feedForward` wird mit einer Ausnahme abgewiesen, was eine robuste API-Nutzung gewährleistet.

### 3.2.3 Bias, Momentum und verwendete Fehlerfunktion

Die Methode `Layer.updateWeights` verarbeitet sowohl die regulären Gewichte als auch die zusätzliche Bias-Spalte (Listing 3.2). Für diese Spalte wird der Eingang schlicht auf 1 gesetzt. Das Array `previousUpdates` speichert für jeden Parameter die letzte Änderung. Die Berechnung von `update` entspricht damit Gleichung (9) unter der negativen Delta-Konvention aus Gleichung (3).

**Begründung:** Die Behandlung des Bias als reguläres Gewicht mit einem konstanten Input von 1 ist eine gängige mathematische Vereinfachung. Code-technisch bedeutet dies, dass keine separate Update-Logik für Bias-Werte programmiert werden muss, was die Fehleranfälligkeit der Implementierung stark reduziert und den Code schlank hält.

```
void updateWeights(double[] deltas, double learningRate, double momentum) {
        int prevSize = lastInputs.length;
        for (int i = 0; i < neurons.length; i++) {
                for (int j = 0; j <= prevSize; j++) {
                        double input = j == prevSize ? 1.0 : lastInputs[j];
                        double update = momentum * previousUpdates[i][j] + learningRate *
 deltas[i] * input;
                        weights[i][j] += update;
                        previousUpdates[i][j] = update;
                    }
            }
    }
```

Die Trainingsroutine ist hardcodiert auf die halbe Fehlerquadratsumme aus Gleichung (2) ausgelegt. Sie nimmt

absichtlich keine austauschbare Fehlerfunktion entgegen, sondern verwendet deren feste Ableitung unmittelbar in `calculateDeltas`. **Begründung:** Für den Scope dieses Projekts minimiert dies die Komplexität. Der ebenfalls vorhandene `MeanSquaredError` dient lediglich der Auswertung und Evaluierung, greift aber nicht in den Gradientenfluss ein.

### 3.2.4 Einbindung in die Trainingsschleife

**Umsetzung:** Die Klasse `App` verwendet ein Netz mit der Topologie 2–4–1, Sigmoid-Aktivierung und dem Zufallsstartwert 42. Die Datei `data/xor.csv` liefert die Trainingsmuster. Die Parameter werden nach *jedem* Muster angepasst (Stochastischer Gradientenabstieg). Der Modusname `sgd` bezeichnet im Projekt somit ein inkrementelles Verfahren mit fester Musterreihenfolge.
**Begründung:** Die feste Musterreihenfolge und der gesetzte Seed (42) dienen primär der Reproduzierbarkeit der Testergebnisse und der sauberen Dokumentation im Rahmen dieses Berichts. Durch den Einsatz des `LearningRateScheduler` im Modus `adaptive` (Halbierung der Lernrate nach 200 Epochen ohne signifikante Verbesserung) wird das Problem der Oszillation in späten Trainingsphasen umgangen, ohne die Kernmechanik der Backpropagation-Gleichungen modifizieren zu müssen.

# 4 Ergebnisse

## 4.1 Korrektheitsprüfung der Backpropagation

Die vorhandene Testklasse `TestFFNeuralNetwork` überprüft die Backpropagation unter anderem mit einem numerischen Gradientenvergleich. Dazu wird jeder Parameter $\theta$, einschließlich der Bias-Werte, einzeln um $\varepsilon = 10^{-6}$ erhöht beziehungsweise verringert. Die zentrale Differenz

$$g_{\text{num}}(\theta) = \frac{E(\theta + \varepsilon) - E(\theta - \varepsilon)}{2\varepsilon} \tag{10}$$

nähert die Ableitung an, während alle anderen Parameter unverändert bleiben. Der Test führt anschließend einen Lernschritt ohne Momentum mit $\eta = 0{,}05$ durch und rekonstruiert den implementierten Gradienten als $g_{\text{impl}} = (\theta_{\text{vorher}} - \theta_{\text{nachher}})/\eta$. Beide Werte müssen bis auf eine absolute Toleranz von $10^{-7}$ übereinstimmen. Damit wird die tatsächliche Parameteränderung gegen eine unabhängig aus Fehlerauswertungen berechnete Näherung geprüft.

Die Gradientenprüfungen umfassen die Aktivierungsfunktionen Linear, Sigmoid, Tanh und ReLU sowie unterschiedliche Topologien. Das Sigmoid-Testnetz 2–3–2–3 prüft insbesondere die Fehlerrückführung über mehrere verborgene Schichten und mehrere Ausgaben. Im ReLU-Test werden positive Eingabesummen gewählt, um die nicht differenzierbare Stelle bei null zu vermeiden. Ein weiterer Test überprüft Momentum über zwei aufeinanderfolgende Lernschritte einschließlich der Bias-Änderungen.

Bei der Prüfung des hier beschriebenen Quellcodestands wurden alle 13 vorhandenen Testgruppen erfolgreich ausgeführt. Dazu gehören auch drei XOR-Tests für die Varianten ohne Momentum, mit Momentum und mit zusätzlicher Lernratenanpassung. Nach jeweils 10 000 Epochen klassifizierten diese Tests alle vier Trainingsmuster korrekt und erfüllten die geprüfte Fehlerschranke $\overline{E} < 0{,}001$. Diese Ergebnisse belegen das Verhalten für die gewählten Testfälle und den festen Startwert 42; sie stellen keinen allgemeinen Konvergenznachweis dar.

## 4.2 Performance und Trainingsverhalten

Um die Performance und Konvergenz der Backpropagation-Implementierung zu bewerten, wurde ein Training des XOR-Problems über 10 000 Epochen im reinen SGD-Modus (ohne Momentum, $\eta = 1{,}0$) durchgeführt. Das Netz mit der Topologie 2–4–1 und dem festen Initialisierungs-Seed 42 zeigte dabei ein stabiles Lernverhalten.

Wie die Messungen belegen, reduzierte sich der mittlere Fehler $\overline{E}$ von initial 0,1398 kontinuierlich auf 0,000063 nach 10 000 Epochen. Die Laufzeit für diesen gesamten Trainingsprozess (inklusive Vorwärts- und Rückwärtsrechnung für alle vier Muster pro Epoche) betrug auf dem Testsystem lediglich 43 ms. Diese geringe Ausführungszeit bestätigt die theoretische Vorüberlegung aus Abschnitt 5, dass die Komplexität pro Muster linear zur Anzahl der Parameter $O(P)$ wächst und die Implementierung in nativem Java (ohne Overhead durch externe Frameworks) für kleine Netztopologien äußerst effizient arbeitet.

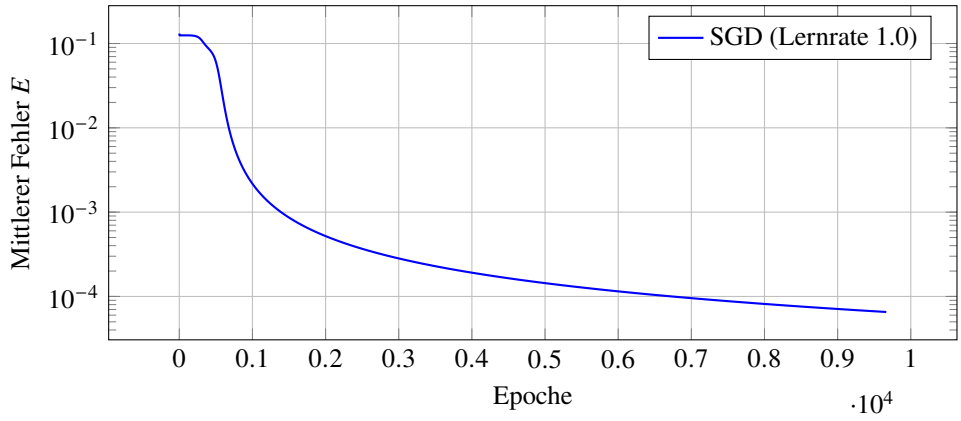
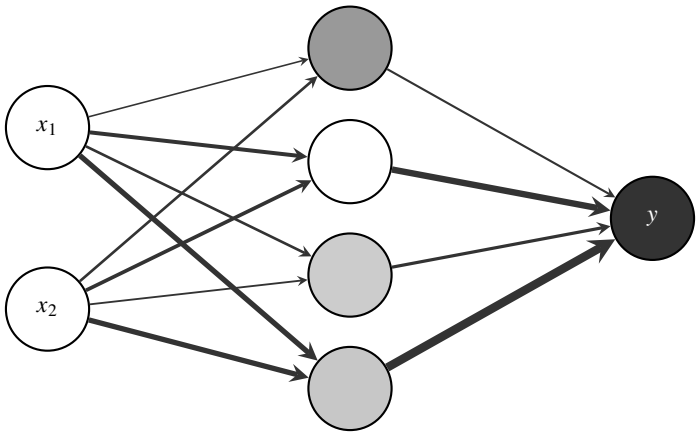



Abbildung 1: Lernverlauf (Loss History) für das XOR-Problem über 10.000 Epochen im SGD-Modus. Dargestellt ist die halbe Fehlerquadratsumme.




| **Schicht / Neuron** | **Gewicht** $x_1$ | **Gewicht** $x_2$ | **Gewicht** $h_1$ | **Gewicht** $h_2$ | **Gewicht** $h_3$ | **Gewicht** $h_4$ | **Bias** $b$ |
|---|---|---|---|---|---|---|---|
| Hidden Neuron 1 | 1,221 | −2,864 | – | – | – | – | −0,911 |
| Hidden Neuron 2 | −4,624 | −4,370 | – | – | – | – | 6,664 |
| Hidden Neuron 3 | −2,850 | −1,736 | – | – | – | – | 3,174 |
| Hidden Neuron 4 | −6,830 | −6,926 | – | – | – | – | 2,897 |
| Output Neuron $y$ | – | – | 2,193 | 8,418 | 3,569 | −11,720 | −5,918 |

Abbildung 2: Visualisierung der 2–4–1 Netzarchitektur sowie Übersicht der gelernten Parameter. Die Liniendicke entspricht dem Betrag der Verbindungsgewichte. Die Graustufe der Neuronen repräsentiert deren Bias-Wert (dunkel = stark negativ, weiß = stark positiv).

# 5 Diskussion

## 5.1 Einordnung der Backpropagation-Implementierung

Die Aufteilung in Vorwärtsrechnung, Delta-Berechnung und Parameterupdate macht die Umsetzung mit den Formeln aus Abschnitt 2.2 vergleichbar. Insbesondere die getrennten Schleifen für Deltas und Gewichtsänderungen stellen einen einheitlichen Parameterstand innerhalb eines Lernschritts sicher. Bei einer Gesamtzahl von $P$ Gewichten einschließlich der Bias-Werte benötigen Vorwärtsrechnung, Rückwärtsrechnung und Update jeweils $O(P)$ Rechenoperationen pro Muster: Jede Verbindung wird pro Durchlauf nur eine konstante Anzahl von Malen verarbeitet. Diese Abschätzung folgt aus den Schleifen im Quellcode; eine Laufzeitmessung ist damit nicht verbunden. Die Gradientenprüfungen aus Abschnitt 4.1 stützen die Korrektheit der getesteten Lernschritte. Die XOR-Tests erfassen darüber hinaus das Zusammenspiel über viele Epochen. Da die Demo jedoch dieselben vier Muster zum Trainieren und zur abschließenden Bewertung verwendet, lässt sich daraus keine allgemeine Aussage über die Generalisierung auf unbekannte Daten ableiten. Für eine solche Bewertung müssten Trainings-, Validierungs- und Testdaten getrennt werden [4, Folie 15].

Eine weitere Grenze ist die feste Kopplung des Trainings an die halbe Fehlerquadratsumme. Eine austauschbare Fehlerfunktion müsste zusätzlich ihre passende Ableitung bereitstellen. Auch ein Vergleich der drei Trainingsvarianten erfordert weitere Experimente: Das Bestehen derselben XOR-Tests allein zeigt noch nicht, welche Variante schneller oder über verschiedene Initialisierungen zuverlässiger lernt.

# 6 Fazit und Ausblick

## 6.1 Zusammenfassung

In diesem Projekt wurde ein mehrschichtiges Feed-Forward-Netzwerk von Grund auf in Java implementiert, ohne auf externe Machine-Learning-Frameworks zurückzugreifen. Die Kernkomponenten umfassen den Vorwärtsdurchlauf, die Fehlerberechnung basierend auf der halben Fehlerquadratsumme sowie die Gewichtsanpassung durch Backpropagation. Zusätzlich wurden Optimierungsverfahren wie Momentum und eine adaptive Lernrate in den Lernprozess integriert.
Die funktionale Korrektheit der Backpropagation-Implementierung konnte durch systematische Tests mit numerischen Gradienten (zentrale Differenz) für verschiedene Aktivierungsfunktionen und Netztopologien erfolgreich nachgewiesen werden. Am Beispiel des XOR-Problems zeigte das Netzwerk mit einer 2-4-1-Architektur ein stabiles und schnelles Konvergenzverhalten. Der mittlere Fehler ließ sich innerhalb von 10.000 Epochen von initial 0,1398 auf 0,000063 minimieren. Die gemessene Laufzeit von lediglich 43 ms für diesen gesamten Trainingsprozess unterstreicht die Effizienz der nativen, schlanken Implementierung. Wie in der Diskussion dargelegt, beschränkt sich die durchgeführte Evaluierung am XOR-Problem jedoch noch auf die Trainingsdaten; ein empirischer Nachweis der Generalisierungsfähigkeit auf unbekannte Testdaten steht aus.

## 6.2 Persönliches Fazit

Die eigenständige Implementierung eines neuronalen Netzes „from scratch“ erwies sich als äußerst wertvolle und lehrreiche Erfahrung. Während moderne Frameworks die komplexen mathematischen Operationen im Hintergrund verbergen, zwang dieses Projekt dazu, sich intensiv mit den Details der Kettenregel, der korrekten Matrix-Indizierung bei den Gewichten und dem genauen Zusammenspiel der Fehlersignale über mehrere Schichten hinweg auseinanderzusetzen. Besonders die saubere algorithmische Trennung von Delta-Berechnung und dem eigentlichen Gewichtsupdate war eine technische Hürde, die das Verständnis für den Datenfluss im Netzwerk deutlich geschärft hat.
Die erfolgreiche Lösung des klassischen XOR-Problems bestätigt, dass die theoretischen Konzepte korrekt in lauffähigen Code überführt wurden. Das übergeordnete Ziel, ein tiefgreifendes mathematisches und programmiertechnisches Verständnis für die Grundlagen des überwachten Lernens zu entwickeln, wurde vollumfänglich erreicht. Das Projekt bildet somit ein sehr solides Fundament, um künftige Problemstellungen auch mit komplexeren Architekturen fundiert bewerten und bearbeiten zu können.

# Literatur


[1] I. Goodfellow, Y. Bengio und A. Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016. URL: https://www.deeplearningbook.org/ (besucht am 27. 10. 2026).

[2] M. A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015. URL: https://neuralnetworksanddeeplearning.com/chap2.html (besucht am 27. 10. 2026).

[3] D. E. Rumelhart, G. E. Hinton und R. J. Williams. „Learning representations by back-propagating errors“. In: *Nature* 323.6088 (1986), S. 533–536. DOI: 10.1038/323533a0.

[4] D. Vieten. „Lernen in kNN: Feed-Forward-Netze und Backpropagation“. Vorlesungsfolien zur Lehrveranstaltung Lernen in natürlichen und künstlichen neuronalen Netzen, Semesterwoche 3, ohne Datumsangabe.